LodgeCore Hospitality Operating System

Master Development Status & Engineering Roadmap

Document Type: Internal Engineering & Project Management Guide

Last Updated: August 2026

---

1. Executive Technical Summary

LodgeCore is being developed as an enterprise-grade, offline-first Hospitality Operating System. It is built as a **Turborepo monorepo** to cleanly separate the web application, database layer, shared types, and hardware agents.

Core Technology Stack

- **Monorepo Manager:** Turborepo (`pnpm`)
- **Web Application (`apps/web`):** Next.js 14+ (App Router), React 19
- **Styling & UI:** Tailwind CSS v4, `shadcn/ui`, `lucide-react`
- **State & Data Fetching:** React Query (v5), Server Actions, `next-auth` (v5 beta)
- **Database (`packages/db`):** PostgreSQL managed via Prisma ORM
- **Validation & Types (`packages/types`):** Zod schema validation
- **Hardware Integration (`hardware-agent` & `apps/desktop`):** C# .NET Desktop Application and Background Agents for local smart lock integration (e.g., Deluns).
- **Background Workers (`workers`):** Edge workers handling external integrations.

---

2. Exhaustive Cross-Stack Audit

This audit evaluates the codebase across four dimensions: **Frontend (UI), Backend (API/DB), Hardware (Local Agents), and Mobile (Guest/Staff Apps).**

2.1 Web Frontend Audit (`apps/web/src/app`)

What has been built in the React / Next.js UI layer.

■ Built & Functional:

- **(frontdesk):** Interactive tape chart (`/frontdesk`), check-in/check-out modals, drag-and-drop reservations, room status indicators, hardware key encoding modals.
- **(dashboard):** Multi-property selector, role-based navigation.
- **(dashboard)/reservations:** Reservation list, guest folios, payment gateways, check-out flows.
- **(dashboard)/rooms & /room-types:** Room inventory management, status overrides.
- **(dashboard)/payments:** Transaction logs, receipt generation.
- **(dashboard)/housekeeping:** Basic housekeeping task list and assignment.
- **(dashboard)/maintenance:** Maintenance ticketing system.

- **(dashboard)/buildings & /properties:** Multi-property structure setup.
- **(dashboard)/night-audit:** Night audit summary and execution dashboard.

■ **Missing from Frontend:**

- **Point of Sale (POS):** No touch-friendly POS interface for restaurants/bars. No Kitchen Display System (KDS).
- **Inventory & Procurement:** No warehouse dashboards, stock count UIs, or Purchase Order forms.
- **Finance & Accounting:** No general ledger, chart of accounts, P&L reporting, or FIRS tax dashboards.
- **HR & Staff:** No scheduling rosters or biometric attendance dashboards.
- **Security:** No visitor log UIs, incident report forms, or generator tracking interfaces.

2.2 Backend & Database Audit (`packages/db` & `apps/web/src/app/api`)

What has been built in the API and Prisma Data Layer.

■ **Built & Functional:**

- **Prisma Models:** 40+ models handling `Organization`, `Property`, `Reservation`, `Room`, `RatePlan`, `Folio`, `Payment`, `DoorLock`, `HousekeepingTask`, `NightAudit`.
- **APIs:** `/api/v1` routes, `/api/auth` (NextAuth setup), `/api/cron` (automated jobs like Night Audit).
- **Server Actions:** Deep integration for tape chart manipulation, check-in flows, and locking operations.

■ **Missing from Backend:**

- **POS Models:** `PosOrder`, `PosProduct`, `PosCategory`, `Recipe`.
- **Inventory Models:** `Warehouse`, `StockItem`, `Supplier`, `PurchaseOrder`.
- **Finance Models:** `LedgerAccount`, `JournalEntry`, `TaxClass` (FIRS).
- **Operations Models:** `GuestComplaint` (SLA), `VisitorLog`, `FuelDelivery`.

2.3 Hardware & IoT Audit (`hardware-agent` & `apps/desktop`)

Local machine integrations for offline operations and external hardware.

■ **Built & Functional:**

- `LodgeCore.HardwareAgent` **(C# .NET):** A background service that runs on the hotel's local server/reception PC. It handles the local network communication with physical door lock encoders (e.g., Deluns locks).
- `LodgeCore.Desktop`**:** A wrapper application to run LodgeCore locally, facilitating the "Offline-First" hybrid cloud architecture.
- **Web Integration:** The web frontend successfully sends commands to the local agent to encode RFID keycards.

■ **Missing from Hardware:**

- **Biometric Scanners:** No SDK integration yet for fingerprint scanners (needed for Staff Attendance).
- **Smart Thermostats:** No integration for energy/ESG reporting.
- **Receipt Printers / Cash Drawers:** ESC/POS integration is required for the upcoming Restaurant POS module.

2.4 Mobile App Audit (Guest & Staff Apps)

Native or hybrid mobile applications.

■ **Completely Missing:**

- There is **no mobile application codebase** in this repository.
- *Requirement:* A LodgeCore Staff App (React Native or Flutter) is required for Housekeeping (to check off rooms), Maintenance (to log faults), and Security (to scan QR code patrol points).
- *Requirement:* A LodgeCore Guest App is required for mobile check-in, digital keys, and room service pre-orders.

---

3. The 5-Phase Execution Roadmap

Based on the exhaustive audit above, here is the strictly sequenced execution plan to complete the LodgeCore Operating System.

Phase 1: Procurement, Inventory & POS (The F&B Engine)

- **Database:** Draft Prisma schema for `Supplier`, `Warehouse`, `StockItem`, `PosProduct`, `PosOrder`, `Recipe`.
- **Backend:** Write TRPC/Server Actions for POS transactions and auto-stock deduction.
- **Frontend:** Build a touch-optimized POS interface (`/pos`) for waitstaff and a KDS interface for the kitchen.

Phase 2: Native Finance & FIRS Compliance (The ERP Engine)

- **Database:** Draft Prisma schema for `LedgerAccount`, `JournalEntry`, `CityLedger`, `TaxClass`.
- **Backend:** Create the automated double-entry journal posting engine (every PMS and POS transaction must create a journal entry).
- **Frontend:** Build the Finance dashboard (`/finance`) showing Trial Balance, P&L, and automated FIRS VAT returns.

Phase 3: Facility Operations (Security, Complaints, Fuel)

- **Database:** Draft Prisma schema for `GuestComplaint`, `VisitorLog`, `FuelDelivery`, `GeneratorRuntime`, `SecurityPatrolLog`.
- **Frontend:** Build management dashboards for security incidents, fuel consumption reports, and SLA-tracked complaints.

Phase 4: HR & Staff Management

- **Database:** Draft Prisma schema for `StaffShift`, `BiometricLog`.
- **Hardware:** Expand `LodgeCore.HardwareAgent` to capture fingerprint inputs.
- **Frontend:** Build the staff scheduling and KPI dashboard (`/hr`).

Phase 5: The Mobile Ecosystem (New Repository)

- **Action:** Initialize a new React Native / Expo repository (e.g., `lodgecore-mobile`).
- **Staff App:** Build the mobile UI for Housekeepers (task lists), Maintenance (ticket logging), and Security (QR patrol scanning).
- **Guest App:** Build the mobile UI for guest check-in, digital tipping, and room service ordering.

---

4. Immediate Next Step

Status: *The project is ready to begin* ***Phase 1****. The absolute first action is to open* `/packages/db/prisma/schema.prisma` *and write the database models for Inventory, Procurement, and Point of Sale.*